



Começando a programar com Qt

Principais classes e conceitos básicos de Qt

Danilo Freire de Souza Santos



Roteiro

- ☐ Hello World
- ☐ Projetos em Qt
- ☐ Signals e Slots
- ☐ Modelo de Objetos

Começando do zero

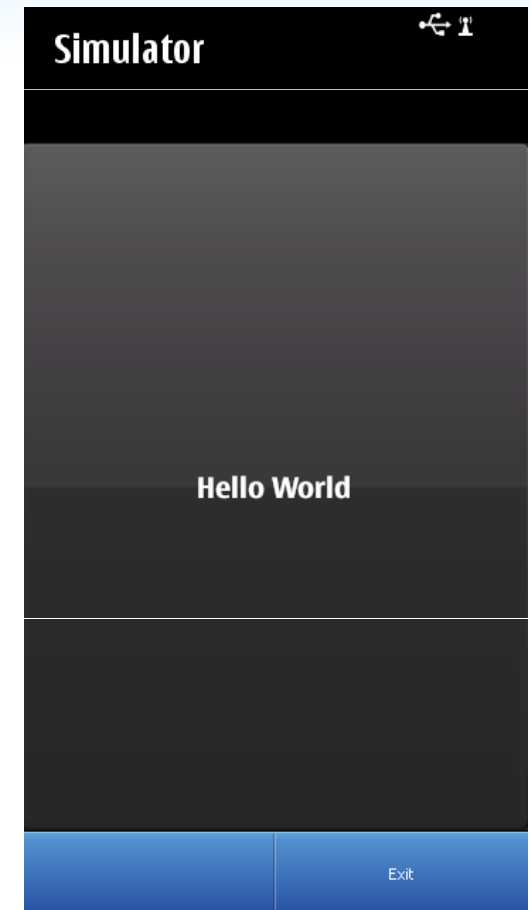
❑ Hello World

```
#include <QApplication>
#include <QLabel>

int main(int argc, char *argv[])
{

    QApplication a(argc, argv);
    QLabel hello("Hello World");
    hello.show();
    //hello.showMaximized();

    return a.exec();
}
```



Explorando o Hello World

```
#include <QApplication>
#include <QLabel>

int main(int argc, char *argv[])
{
    QApplication a(argc, argv);
    QLabel hello("Hello World");
    hello.show();
    //hello.showMaximized();

    return a.exec();
}
```

includes

QApplication

Label com a
string

Mostre o Label

Execute a
aplicação

Arquivos de Projeto

❑ Arquivo .pro

- Arquivo com definições do projeto

❑ Define variáveis para o compilador

- Arquivos fontes, includes, dependências, etc...

❑ Oferece templates

- app: cria um makefile para aplicações
- lib: cria um makefile para bibliotecas

❑ Define escopos

- Uma definição para Symbian, outra para Desktop, outra para Maemo...

```
QT += core gui
TARGET = mymainwindow
CONFIG += console

TEMPLATE = app

SOURCES += main.cpp \
mainwindow.cpp

HEADERS += \ mymainwindow.h

symbian {
    TARGET.UID3 = 0xE515E66B
    vendorinfo = \ "%{\\"Nokia\\"}" \ ":"\\"Nokia\\"""
    my_deployment.pkg_prerules = vendorinfo
    DEPLOYMENT += my_deployment
}
```

qmake e build...

☐ qmake é o “pré-compilador” do Qt

- Gera os arquivos makefile para o compilador C++
- Gera outros arquivos fontes (Meta-Object)
- Utiliza como base o arquivo de projeto .pro
- `/> qmake`

☐ Depois de executar o qmake, chame o make

- `/> make`

☐ Não se preocupem, o QtCreator gerencia isso para você!

Executando no Qt Creator

- ❑ Usando o Qt Simulator
- ❑ Sigam as instruções do Professor!



Executando em Symbian (no aparelho)

☐ Pré-requisitos

- Instalar o Nokia Ovi Suite no Computador
- Instalar o Nokia TRK no Celular
 - Pacote de instalação no Qt SDK

☐ Conecte o aparelho via USB

☐ Mude o “target” de compilação para Symbian

- Siga instruções do professor!

☐ Observe se o Qt Creator identificou o aparelho

☐ Run!



Explorando o projeto Symbian

☐ Arquivos de compilação Symbian são criados automaticamente pelo Qt SDK (qmake, etc...)

☐ São gerados:

- Arquivos pkg para geração de instaladores
- Arquivos mmp, que são os arquivos “makefile” do symbian
- Arquivos de recursos, etc

☐ Sigam as instruções do professor

Organizando em Layouts

❑ Incrementando o Hello World:

```
#include <QApplication>
#include <QPushButton>
#include <QVBoxLayout>
#include <QSpinBox>
#include <QSlider>

int main(int argc, char *argv[])
{
    QApplication a(argc, argv);
    QWidget window;
    QVBoxLayout* layout = new QVBoxLayout(&window);
    QSpinBox* spinBox = new QSpinBox();
    QSlider* slider = new QSlider(Qt::Horizontal);
    QPushButton hello = new QPushButton("Hello World");
    layout->addWidget(spinBox);
    layout->addWidget(slider);
    layout->addWidget(hello);
    window.showFullscreen();
    return a.exec();
}
```

Dividindo a tela em um layout vertical

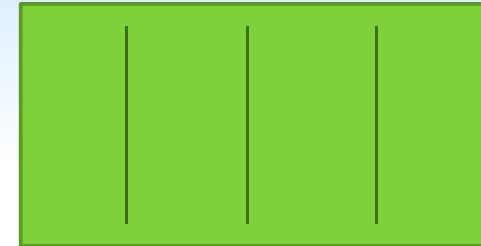
Criando novos elementos
que derivam de QWidget

Adicionando os widgets no layout

Layouts mais comuns

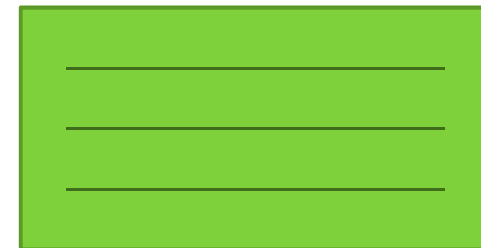
❑ QHBoxLayout:

- Layout Horizontal



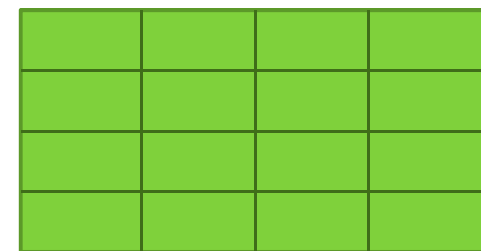
❑ QVBoxLayout:

- Layout Vertical



❑ QGridLayout:

- Grade



❑ QFormLayout:

- Associa widgets de input e labels



Criando Conexões

❑ Vamos adicionar funcionalidade a aplicação.

- Caso de uso: A aplicação está em Fullscreen(), precisamos sair dela.

❑ Como faz?

- Usar Signals e Slots:

```
QObject::connect(hello, SIGNAL(clicked()), &a, SLOT( quit() ) ) ;
```

❑ Interpretando:

- Quando o botão “hello” for clicado, o QApplication “a” irá sair.

Callbacks X Signals & Slots

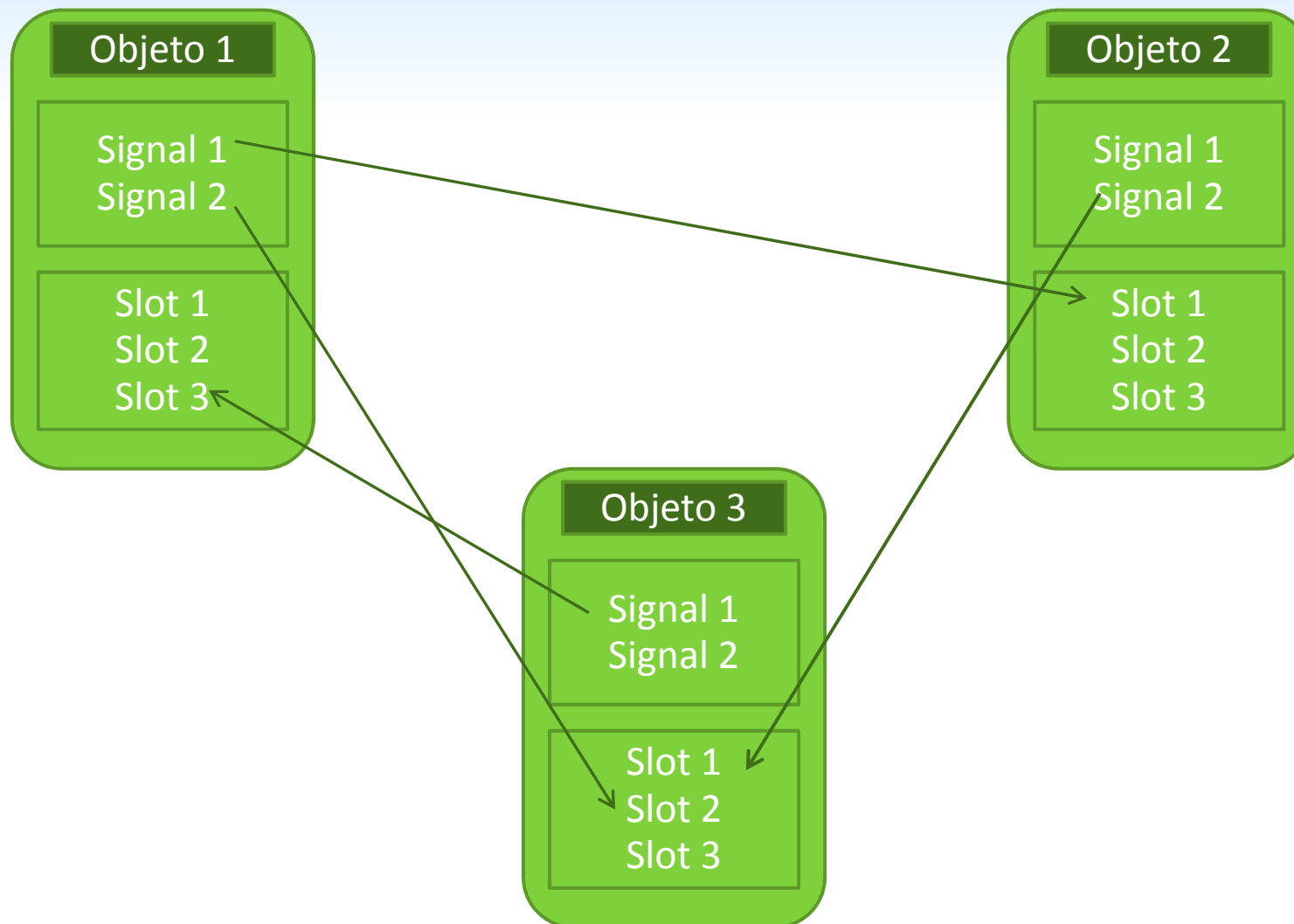
Callbacks

- Geralmente são ponteiros para funções
- São pouco flexíveis
 - As funções devem ser casadas em tempo de compilação
 - Não garantem a segurança de tipo (*void)

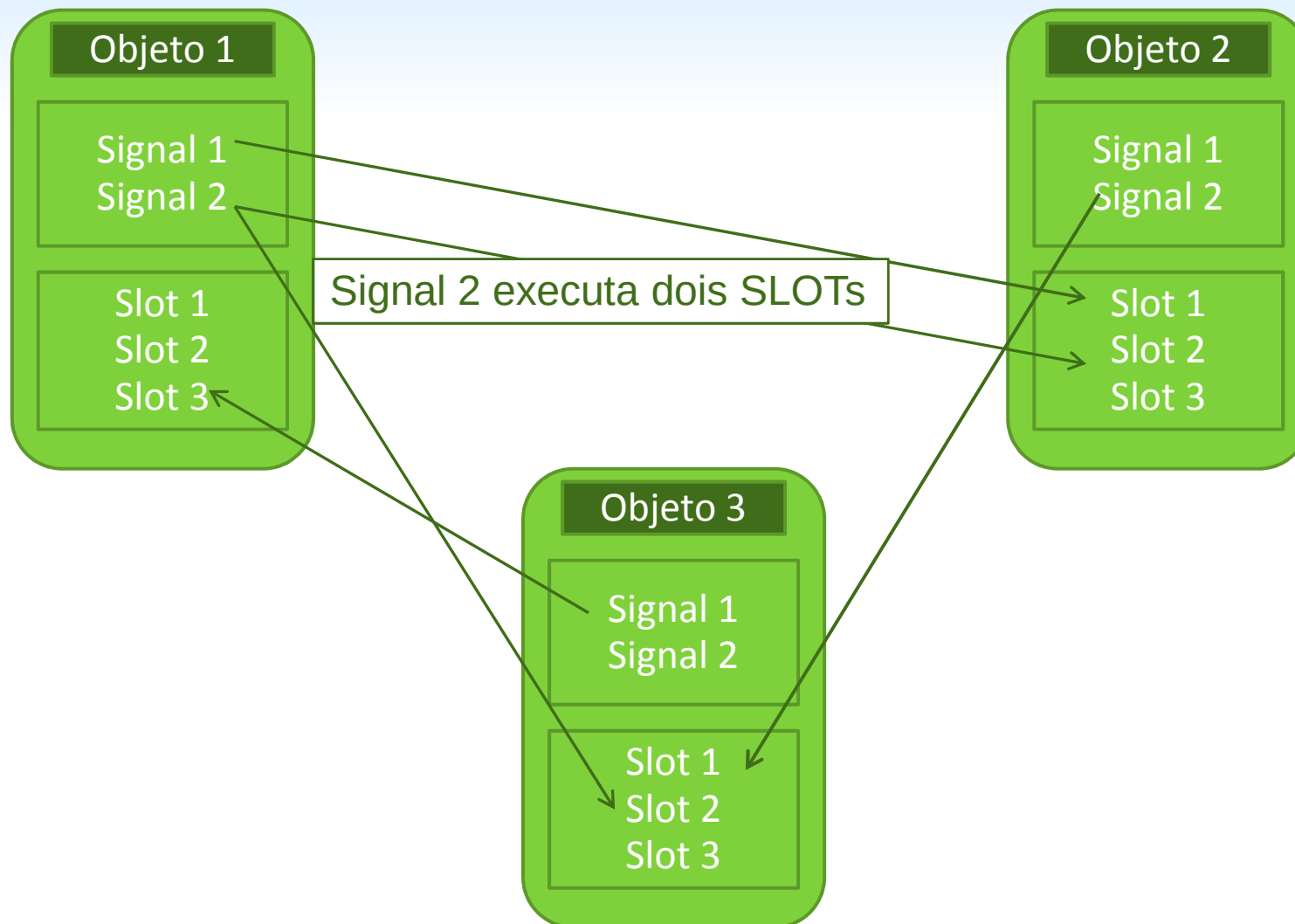
Signals & Slots

- Separação:
 - Signal: Emitido para lançar um evento
 - Slot: Uma função que pode ser ligada em um signal, ou seja, a ação para um evento
- Garantem a conexão entre tipos (type-safe)
- Vários objetos Qt já oferecem seus Signals e Slots

Conexões...



Múltiplas Conexões



Exemplo – múltiplas conexões

```
#include <QApplication>
#include <QPushButton>
#include <QVBoxLayout>
#include <QSpinBox>
#include <QSlider>

int main(int argc, char *argv[]) {
    QApplication a(argc, argv);
    QWidget window;
    QVBoxLayout* layout = new QVBoxLayout(&window);
    QSpinBox* spinBox = new QSpinBox();
    QSlider* slider = new QSlider(Qt::Horizontal);
    QPushButton hello = new QPushButton("Hello World");
    layout->addWidget(spinBox);
    layout->addWidget(slider);
    layout->addWidget(hello);
    window.showFullscreen();
    QObject::connect( hello, SIGNAL( clicked() ) ),
        &a, SLOT( aboutQt() ) );
    QObject::connect( hello, SIGNAL( clicked() ) ),
        &window, SLOT( showMaximized() ) );
    return a.exec();
}
```


Parâmetros em Signals

☐ É possível transmitir parâmetros em sinais

- `QObject::connect( spinBox, SIGNAL( valueChanged(int) ), slider, SLOT( setValue(int) ) );`

☐ O tipo do parâmetro do signal deve “casar” com o do slot

- Se não existir um Signal compatível com o SLOT, não ocorrerá erro de Compilação!
- Entretanto, um warning em tempo de execução aparecerá.

☐ Porque é type-safe?

- Sem conexão se os tipos não casarem ou se o signal ou slot não existir
- O método `connect()` retorna um boolean indicando o sucesso

Exemplo – Parâmetro em Signals

```
#include <QApplication>
#include <QPushButton>
#include <QVBoxLayout>
#include <QSpinBox>
#include <QSlider>

int main(int argc, char *argv[])
{
    QApplication a(argc, argv);
    QWidget window;
    QVBoxLayout* layout = new QVBoxLayout(&window);
    QSpinBox* spinBox = new QSpinBox();
    QSlider* slider = new QSlider(Qt::Horizontal);
    QPushButton hello = new QPushButton("Hello World");
    layout->addWidget(spinBox);
    layout->addWidget(slider);
    layout->addWidget(hello);
    window.showFullscreen();
    QObject::connect( spinBox, SIGNAL( valueChanged(int) ),
                     slider, SLOT( setValue(int) ) );
    return a.exec();
}
```

Exercício 2

❑ Tomem como base o último exemplo!

- Crie um widget com um slider e um spinbox.
- Quando o slider alterar o valor, alterem o valor no spinbox, e vice e versa.

Qt Object Model

- ❑ **QObject** estende as funcionalidades de Classes C++
- ❑ **Adiciona mais flexibilidade e preserva a eficiência de C++**
 - Gerenciamento de Memória
 - Signals e Slots
 - Propriedades e meta-information
 - Introspecção em C++

QObject

❑ Principal classe de Qt

- Necessária para fazer uso de meta-information

```
#include <QObject>

class MyClass: public QObject
{
    Q_OBJECT
    ...
};
```

❑ “moc” – Meta-Object Compiler

- Interpreta QObjects e estende o código fonte com funções extras
- Remove palavras chaves de Qt (signals, slot, emit), e gera um código fonte C++ padrão

❑ Funciona com qualquer compilador padrão

Mais Signals e Slots

☐ Tipos de conexão:

- Direta: Conexão padrão, o Slot é executado imediatamente depois do Signal
- Queued: O Slot é executado posteriormente (na volta do event loop de execução de Qt)

☐ Implementação

- Signals:
 - Gerados automaticamente pelo moc
 - Definam no .h (nunca no .cpp).
 - Não retorna valor
- Slots:
 - Podem ser virtuais, nunca estáticos
 - Oferecem um pouco mais de sobrecarga em relação a um método normal

Implementando seus Signals e Slots

☐ Criar um QObject Contador com:

- 1 SLOT: função que atribui um valor a uma variável
- 1 SIGNAL: sinal que é emitido quando a variável é alterada

☐ Crie um QPushButton

- Conecte o signal clicked() ao SLOT do contador

☐ Crie um QLabel

- Conecte o Signal de Contador ao SLOT setNum() do QLabel

☐ Incluem todos os objetos em uma aplicação.

Meta-Object Compiler

❑ Geração dos arquivos moc_...

❑ O que o moc nos oferece:

- Signals and Slots
- `metaObject()`: retorna o meta-object para a classe
- `QMetaObject::className()`: retorna o nome da classe em tempo de execução
- `inherits()`: checa a instância atual herda de outra classe
- `tr()`: traduz strings
- `setProperty()` e `property()`: acessa e atribui valores a propriedades dinamicamente
- Mecanismos mais seguros para casting:
 - `qobject_cast<>`

Gerenciamento de Memória

❑ **QObject implementa uma hierarquia pai-filho**

- Ao criar um QObject com a referência do pai, o pai adiciona o objeto a sua lista de filhos
- Quando o pai for deletado, todos os filhos são deletados automaticamente
- Se o filho for deletado, ele é removido da lista do pai automaticamente
- CUIDADO: Apenas são gerenciados objetos criados com a referência do pai!

❑ **Em relação aos widgets**

- Os widgets filhos são exibidos dentro da área do widget pai.

Exemplo

```
QWidget* win = new QWidget();  
QVBoxLayout* layout = new QVBoxLayout(win);  
QPushButton* botao = new QPushButton("Label");  
layout->addWidget(botao);  
win->show();
```

QPushButton é filho de quem?

```
QWidget* win = new QWidget();  
QVBoxLayout* layout = new QVBoxLayout(win);  
QPushButton* but = new QPushButton("Label");  
layout->addWidget(but);  
win->show();  
win->dumpObjectTree();
```